

System Calls in Operating System

1. How do apps interact with the Kernel? → Using System Calls

Applications cannot touch the kernel directly.

They always use **system calls** to request services from the operating system.

Example 1: mkdir laks

When you run:

```
mkdir laks
```

this is what actually happens:

- **mkdir indirectly calls the kernel.**

It asks the kernel's file-management module to create a new directory.

- **mkdir is just a wrapper.**

The real work is done by the **actual system calls** inside the kernel.

- **So, mkdir interacts with the kernel through system calls.**

Example 2: Creating a Process

When a user starts a program:

Steps:

1. **User executes a process**
(This happens in *User Space*.)
2. **A system call is needed**
for example, to create a new process (fork, exec).
3. **The system call enters Kernel Space**
Here, the kernel creates the process.
4. **Control returns back to User Space.**

2. How does the switch from User Space to Kernel Space happen?

→ **Using software interrupts (also called traps).**

These are special CPU instructions that safely transfer control to the kernel.

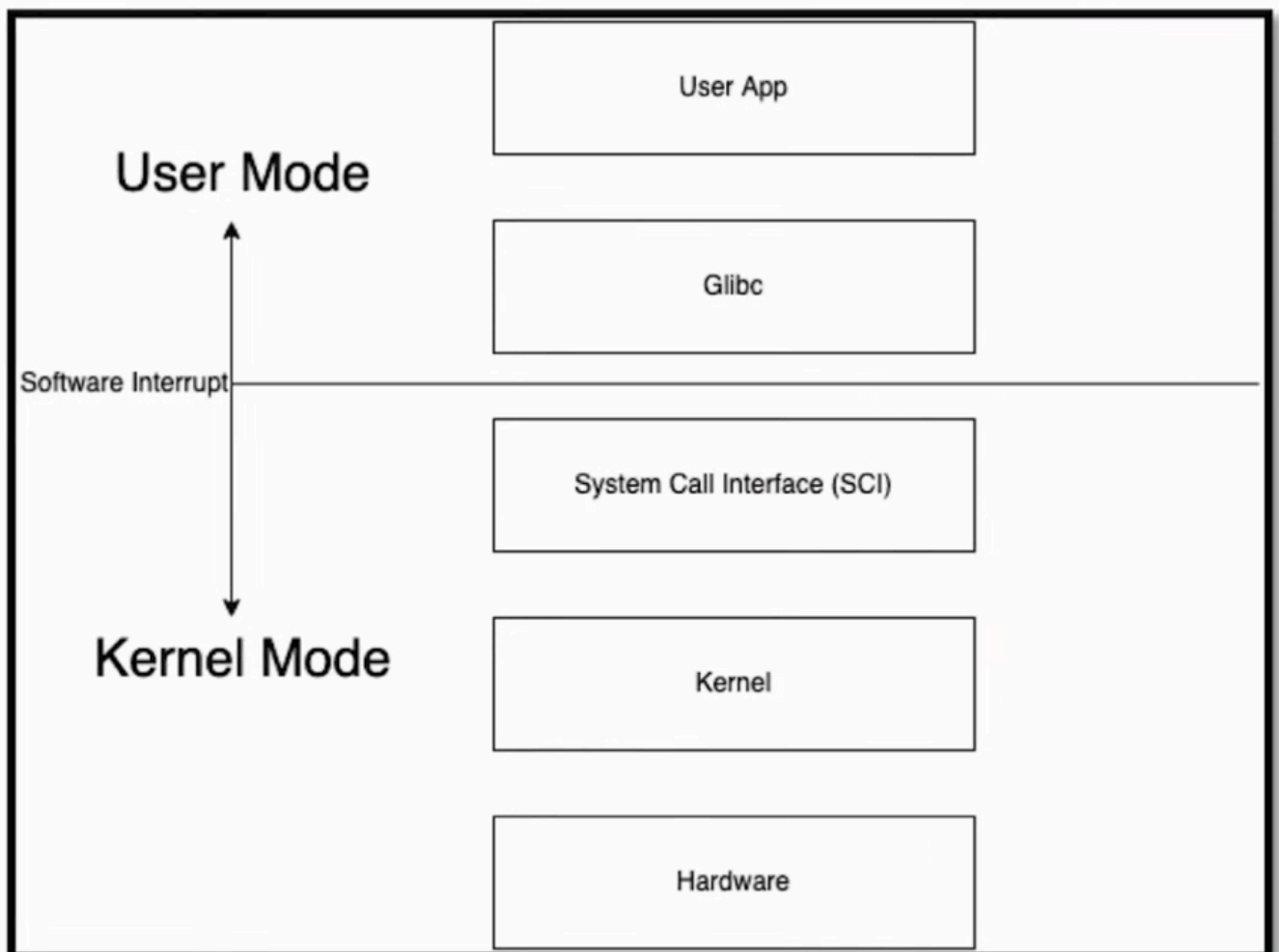
System calls are implemented in C

Most system calls inside the kernel are written in the C language because C gives low-level control and has been traditionally used for OS development.

A system call is the bridge between user space and kernel space.

It allows applications to ask the kernel for services safely.

System call are the only way through which process can go into kernel mode to user mode.



Types of System Calls:

- 1) Process Control
 - a. end, abort
 - b. load, execute
 - c. create process, terminate process
 - d. get process attributes, set process attributes
 - e. wait for time
 - f. wait event, signal event
 - g. allocate and free memory
- 2) File Management
 - a. create file, delete file
 - b. open, close
 - c. read, write, reposition
 - d. get file attributes, set file attributes
- 3) Device Management
 - a. request device, release device
 - b. read, write, reposition
 - c. get device attributes, set device attributes
 - d. logically attach or detach devices
- 4) Information maintenance
 - a. get time or date, set time or date
 - b. get system data, set system data
 - c. get process, file, or device attributes
 - d. set process, file, or device attributes

Examples of Windows & Unix System calls:

Category	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle() SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	open () read () write () close () chmod() umask(chown()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Management	GetCurrentProcessID() SetTimer() Sleep()	getpid () alarm () sleep ()